

Development Manual

Architecture, conventions and extension

Transit Calculator

Version 1.0 · 14 August 2026

Who this is for

Anyone changing the code. Assumes Python and a working knowledge of the domain; read the Technical Manual first for why the algorithms are what they are.

1 Shape of the thing

A local HTTP server and a single-page client. The browser owns the map, the drawing and the table. The server exists for the three things a page cannot do for itself: proxy chart tiles past CORS, decode binary forecast files, and read and write files on disk.

Standard library only, with one optional exception (certifi, for a single TLS chain). That constraint is deliberate — the tool has to run from a cold machine at sea with nothing to install.

Module	Responsibility	Depends on
geo.py	Vincenty geodesics, UTM conversion, set and drift. No I/O, no state.	math
transit.py	The marching calculation	geo
fuel.py	The vessel fuel chain	model.json
ofs.py	Forecast-model domains, selection, chaining	currents, geo
currents.py	OPeNDAP client and cycle cache (vendored)	—
marine.py	Buoy and forecast sources, IDW blender	geo
charts.py	Tile fetch, cache, prefetch	—
shapefile_io.py	Shapefile read and write	struct
exporters.py	The output formats	geo, shapefile_io
server.py	HTTP surface, request assembly	all of the above
static/transit.html	The entire client	—

2 Conventions

Get these wrong and everything still runs

Each of these is a silent-failure mode: the code keeps working and the numbers are wrong.

Convention	Rule
Bearings	0 = north, clockwise, degrees true
Current set	The direction the water flows TOWARD (oceanographic), the opposite of the wind convention. Decided in one place, <code>geo.uv_to_set</code> , and asserted on all four quadrants.
Wind direction	The direction it comes FROM. A <code>wind_from</code> equal to the course is a HEADWIND. A test once asserted the opposite of this and passed, because a multi-course line hid the sign.
Shapefile geometry	(x, y) = (longitude, latitude)
GeoJSON / KML / GPX	WGS84 longitude/latitude by definition, so a CRS choice applies to the shapefile only
<code>inverse()</code> third value	The forward azimuth at the destination, not the reverse azimuth

3 Testing

```
python tests/test_transit.py
python tests/mutate.py
```

371 checks; 58 mutations, all of which must be caught.

3.1 Two rules the suite is written to

- A TEST IS NOT TRUSTED UNTIL IT HAS FAILED ON PURPOSE. `mutate.py` breaks one behaviour at a time and confirms the suite goes red. It works on a TEMP COPY — the real source is opened read-only and never written, because a killed mutation runner that edits real files leaves the source corrupted.
- EVERY REFUSAL IS PAIRED WITH AN ACCEPTANCE. A test that only proves the code says no is satisfied by code that always says no.

3.2 Checks can be decorative, and mutation is how you find out

Two checks in this suite passed for the wrong reason until mutation testing exposed them: one exercised an unreachable branch, and one compared against a fallback table identical to the real one, so a function that ignored the real table entirely still passed. Both are now real. If a mutation is MISSED, the check it should have broken is decorative — fix the check, not the mutation.

3.3 Do not assert on incidental data

One check required the whole passage to differ from the benign time by more than three minutes. It passed only because of which forecast cycle happened to be cached: on a different tide phase the fair and foul legs cancel and the total lands close to benign with nothing wrong. The total is a function of departure phase; the per-leg effect is not. If a check depends on which file is in a cache, it is testing the cache.

4 Extending it

4.1 A different vessel

Replace `model.json`. The schema is speed-versus-RPM and fuel-versus-RPM per configuration, plus tank volume, reserve policy, and the two environmental premiums. Nothing else needs to change. Note that the fuel suite asserts its fitted anchors only when the model is not a placeholder — see the public export.

4.2 Another forecast model

Add an entry to `ofs.REGISTRY` with a nominal bounding box and a rank (finer regional models rank lower and win in overlaps). The true domain is probed and cached on first use. The reader assumes the CO-OPS regulargrid layout and reads grid dimensions from the dataset's DDS.

4.3 Another weather source

Return samples shaped like the existing ones — position, source name, and a values dict carrying whatever fields that source actually has. The IDW blender is source-agnostic and per-variable, so a source that supplies only one field is a first-class citizen. Add a quality weight to `marine.QUALITY`.

4.4 Another export format

Add a writer and an entry in `exporters.FORMATS`. The dispatch test walks that list, so a new format is covered the moment it is registered; an unknown format must raise rather than silently defaulting.

5 Publishing

Never flip the private repository to public

Its history carries the measured vessel model, and history is permanent once cloned. Publishing is an EXPORT to a separate repo, built only by tools/make_public.py: archive the tracked tree, swap the vessel for a synthetic placeholder, genericise identifiers, and REFUSE to finish if anything forbidden survives.

Every difference between public and private must live in that script, or the next export silently reverts it.

5.1 What the guard learned the hard way

- The script genericised ITSELF, rewriting its own substitution table. It now drops itself from the export.
- It scrubbed the vessel's NAME and published the measured NUMBERS. The name was never the sensitive part.
- An email address in a User-Agent shipped in the first publish. Any email address is now forbidden.
- Absolute machine paths named private sibling projects and pointed a cloner at a drive they do not have.

The lesson generalises: the guard checked what was expected to be sensitive and nothing else. Ask what is in a file that is not the code.

6 Documents

```
python tools/build_figures.py
```

```
python tools/build_docs.py
```

Figures first — the manuals embed them. Both are reproducible: set SOURCE_DATE to keep the title-page date stable across rebuilds.

Figures are generated FROM THE CODE wherever possible: the marching figure calls the planner, the domain figure reads the probed domains, the direction figure runs both directions. A hand-drawn diagram of what the code is believed to do is a diagram of a belief. These go stale loudly, by failing to build, rather than quietly by being wrong. The same applies to the numbers in the text, which are read at build time.

A document builder is unverified until you look at the pages

Rasterise the output and read it. Two figures in this set had text colliding with a title and an arrow struck through a label — both invisible in the source and obvious on the page.

7 Known gaps

- The forecast-model registry is hand-listed; models outside it are never considered, and there is no global fallback, so a line beyond every regional domain still reports a gap.
- ENC vector features are not ported from the companion console — only the raster background. Clearance-checking against charted hazards would start there.
- No route optimisation: the tool computes the line it is given.
- `weather.py` is superseded by `marine.py` for the fetch path but still serves one endpoint.
- The per-leg shapefile export truncates the ETA string to the dBASE field width, losing sub-second precision.